



Ben-Gurion University of the Negev

Faculty of Engineering Sciences

Department of Electrical and Computer Engineering

Introduction to Digital Image Processing 361-1-4751

Final Project Report - Backgammon

Submitted to: Prof. Tammy Riklin Raviv

Submitted by: Ori Shlomi, Raz Barak and Gilad Biton

ID Number: 205880206, 209005727, 206544389

Email: orishlo@post.bgu.ac.il , rab@post.bgu.ac.il , giladbit@post.bgu.ac.il

Submission Date: 24/03/2025

Table of Contents

ABSTRACT	2
SYSTEM DESCRIPTION.....	3
INTRODUCTION	3
SETUP	3
ASSUMPTIONS.....	4
CHALLENGES	4
RESTRICTIONS	5
GAME LOGIC AND BACKGAMMON RULES	5
SYSTEM FLOW	6
FLOWCHART – REAL-TIME FRAME ANALYSIS IN THE GAME	7
GAME ANALYSIS.....	8
BOARD DETECTION	9
GRID CREATION	15
CHECKERS DETECTION.....	21
DICE DETECTION	27
HAND DETECTION	32
ALGORITHMS ANALYSIS	35
GAUSSIAN AND MEDIAN BLUR	35
CANNY EDGE DETECTION.....	37
DILATION	39
EROSION	39
MORPHOLOGICAL OPERATION – CLOSING.....	40
FIND CONTOURS (CONTOUR DETECTION)	41
APPROXIMATE POLYGON	42
TEMPLATE MATCHING	43
AFFINE & PROJECTIVE TRANSFORMS	45
BOUNDING BOX.....	47
DBSCAN CLUSTERING.....	48
OPTICAL FLOW – FARNEBACK’S ALGORITHM	50
SSIM SCORE.....	52
ANALYSIS – DICE NUMBER DETECTION	53
CONCLUSIONS.....	55
BIBLIOGRAPHY	57

Abstract

In this project, we developed a computer vision-based system for real-time monitoring and recognition of a physical backgammon game, without requiring any interaction from the players. Our goal was to fully detect the board state- including the position of all checkers, identification of the dice and their values, and monitoring hand movements- to understand and validate moves according to the rules of the game. Throughout development, we faced several significant challenges: a glossy lacquered board that caused problematic light reflections, board color like human skin tones which complicated hand segmentation, white dice resembling the white checkers, and a table surface nearly identical in color to the game board-making contour detection more difficult. In addition, fast hand movement and the variable orientation of the dice during throws required robust and adaptive solutions.

The system is built upon a multi-stage processing pipeline that includes:

- **Board Detection** – Locating the board in the image and aligning it for top-down analysis.
- **Grid Construction** – Dividing the board into 24 playable zones based on triangle detection.
- **Checker Detection** – Color segmentation, template matching, and DBSCAN clustering to accurately locate checkers.
- **Dice Detection** – Separating dice from white checkers, aligning them, and detecting dot patterns using template matching and clustering.
- **Hand Detection** – Computing optical flow between frames to identify motion and locate hands.
- **Move Validation** – Comparing board states and checking legality based on official game rules.

The result is a robust, real-time system that tracks game progress accurately, applying key image processing techniques learned in the course through thoughtful design and implementation.

This report presents the system design, key implementation steps, and the reasoning behind our choices, highlighting how we addressed challenges to build a reliable solution.

YouTube link for a small demonstration of the project:

<https://www.youtube.com/watch?v=p1ABqMCcMII&feature=youtu.be>

System Description

This section provides an overview of the main components of our system. It begins with the physical setup, including the game board, camera, and coding environment. Then, it outlines the overall workflow of the system during gameplay. We also describe the key technical challenges we faced during development, define the necessary constraints for proper system operation and the assumptions made to support our algorithm design and implementation.

Introduction

This project focuses on the real-time detection and tracking of a physical backgammon game using computer vision techniques. The goal is to develop a system that can automatically monitor the gameplay, identify the state of the board, and determine legal moves without requiring any interaction from the players. To achieve this, we designed a vision-based setup that captures the game from a top-down view using a single fixed camera mounted on a custom-built tripod. This camera records the board throughout the game and provides the video feed that serves as input for our image processing pipeline. The system detects the game board, segments its triangle into a grid of playable tiles, recognizes the position of each checker and dice, and analyzes player interactions. Beyond just detecting physical objects, our system integrates logic validation based on the rules of backgammon to ensure consistency between visual input and the expected game flow. The entire project is implemented in Python, leveraging image processing tools to bridge the gap between physical gameplay and digital interpretation.

Setup

1. Physical backgammon game board with 15 checkers (game soldiers) per player with 2 dice.
2. Self-built camera tripod.
3. Given camera by the course with a resolution of 1280x720 and 7.5 fps (default value).
4. Code setup - All the code was written in python using open-cv and scikit libraries.



board



checkers



dice



camera



Camera stand

Assumptions

To simplify the system's design and improve its reliability, we made several key assumptions about the environment and setup. These assumptions allowed us to develop more focused algorithms and avoid overcomplicating the detection process.

The following assumptions were considered throughout the development of the project:

1. Normal room lighting without significantly reflecting light spots on the game board.
2. The camera is set on the tripod above the game board (it **can** be set at an angle).
3. When creating the grid, the board must be clear of checkers and dice.
4. All the game board corners must be visible in the frame (can be covered by hands but not placed outside the frame).
5. In each given frame, the game board is the largest quadrilateral in the image.
6. A “legitimate frame” is a frame where the game board was detected hence the game pieces (checkers and dice) will always be of the same size proportional to the game board. Only these frames will be further processed to detect the game pieces.

Challenges

During development and real-time operation, the system faced several technical and environmental challenges. These obstacles affected both the accuracy and robustness of detection and tracking which led us to optimize our algorithms.

The following list outlines the key challenges we encountered:

1. Lighting might change between each frame taken.
2. The board position might vary during the gameplay.
3. The table the game board is placed on can vary in color and texture.
4. Chosen game board:
 - a. The game board has a lacquer finish which reflects light instead of absorbing it as in plain wood.
 - b. The checkers have a unique texture on the top surface and aren't flat creating different shades and textures on each checker depending on the lighting and the position of the checker in the game board (Show in Image).

5. The board color is like human skin color making it difficult to segment the hand from the board based on a color space.
6. Player's hands move at a fast rate.
7. Build the full logic of backgammon to make the application real time relevant.

Restrictions

To ensure the system operates correctly and maintains both logical and visual consistency, we defined several operational restrictions. These restrictions are intended to reduce detection errors and allow the system to reliably follow the game's progression. Users are expected to follow these guidelines to avoid interfering with the game's detection process.

Below are the restrictions we established:

1. To initiate the game, the board must be well defined without objects touching it.
2. At the end of each turn the game board must be completely visible without any hands interfering.
3. Logical - At player X turn, player Y is eligible to take the dice but not eligible to throw them before player X turn has ended.
4. Logical - The application does not detect Cheating (moving in the wrong direction, etc.).

Game Logic and Backgammon Rules

The system integrates a logic module based on the official rules of backgammon, aimed at validating each move during gameplay. For every frame captured by the camera, the system analyzes the current board state, identifies the position of all checkers and dice number, and determines whether the change from the previous frame constitutes a legal move.

What Is a Legal Move?

A move is considered legal if all the following conditions are met:

1. **Player Turn:** Only the player whose turn it may move checkers.
2. **Dice Usage:** The move must match the values shown on the dice.
3. **Valid Movement:**
 - Checkers move in the correct direction (from the opponent's home to the player's home).

- A checker can land on an empty point, a point with the player's own checkers, or a point with only one opponent checker (which can be "hit").
 - If both dice can be played, both must be used (unless there is no legal option).
 - If a double is rolled (e.g. 3-3), the player must play the number four times.
4. Bar Rule: If a player has one or more checkers on the bar (after being hit), they must re-enter those checkers before moving others.
 5. Bearing Off: Checkers may be born off (removed from the board) only if all of a player's checkers are in their home quadrant. The checker must match the exact dice value or be the furthest one if no match exists.

System Flow

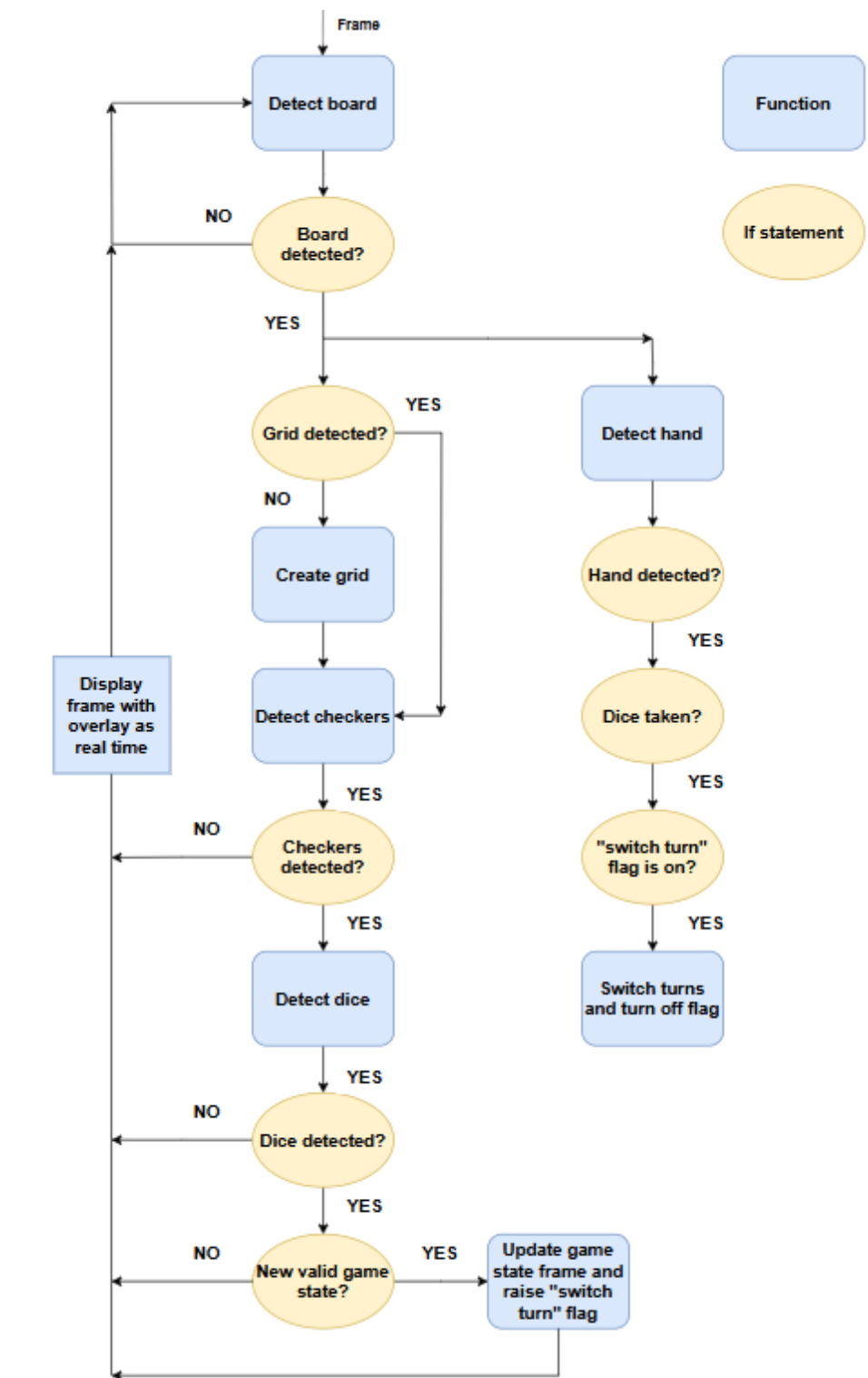
The process begins with detecting the game board and setting up the initial state of the game. From that point on, the system continuously analyzes each frame captured by the camera. In every frame, the board is detected, the positions of the checkers and dice are identified, and the performed move is logically verified according to the rules of the game.

The following is a detailed breakdown of the system's main operational steps:

1. Detect game board.
2. Create game board grid matrix
3. Setup initial checkers position.
4. Decide which player starts - each player throws the dice and the player with the higher value starts (the code takes in consideration receiving the same value and rethrowing the dice).
5. Detect a given turn being played. from now on, for each frame:
 - a. Detect game board.
 - b. Detect checkers and their position.
 - c. Detect dice and their values.
 - d. Check logically if the given game state is the performed move, if yes switch turn and update game state.

Flowchart – Real-Time Frame Analysis in the Game

The following flowchart showcases the internal logic of the system and how it processes each frame. It visually presents the sequence of operations performed by the system to detect game elements, validate moves, and handle turn switching during gameplay.



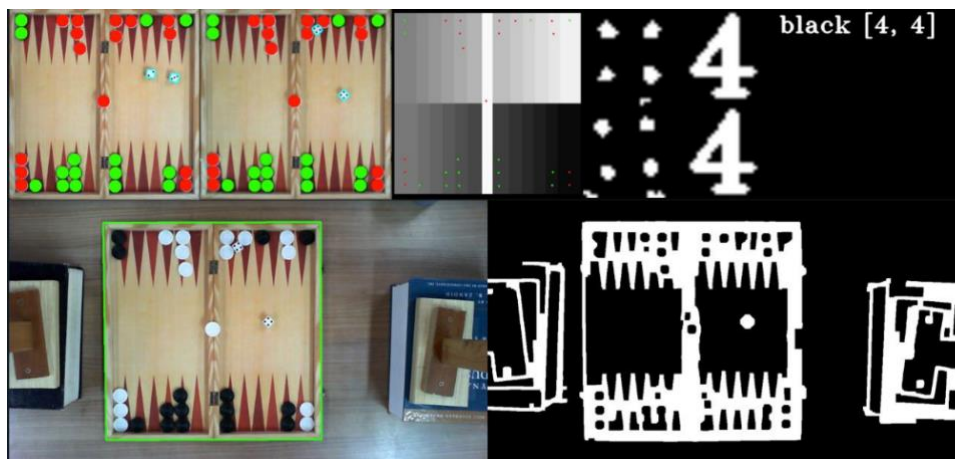
Game Analysis

In this section we will describe our computer vision algorithms that build the game flow. To ensure accurate and stable detection, we follow a structured, step-by-step process where each stage builds upon the previous one, allowing the system to handle lighting changes, partial obstructions, reflections, and perspective variations. At the heart of the system is a methodical approach that segments the game into distinct components, making it possible to track every move and detect any changes that occur on the board.

The game analysis section consists of five main stages:

1. **Board Detection** - Identifying the board's frame and adjusting the perspective to maintain a consistent viewing angle and define the game area (range of interest (ROI) of the frame).
2. **Grid Creation** - Dividing the board into 24 distinct playable areas, defining their boundaries, and marking the bar section.
3. **Checker Detection** - Locating all checkers on the board based on their color and shape and mapping their positions onto the grid.
4. **Dice Detection** - Identifying the dice on the board, detecting the number of dots on each die, and updating the game state accordingly.
5. **Hand Detection** - Recognizing player hand movements during dice rolls, tracking when the hand covers and releases the dice.

The combination of these steps ensures precise and reliable game tracking, maintaining consistency over time. The system captures every change on the board in real time, ensuring that all interactions are accurately recognized and the game progresses smoothly without interruptions.



Board Detection

General Information

Board detection is a fundamental step in the system focusing the frame into our ROI, serving as the primary reference point for tracking all game elements, including checker positions, dice recognition, and hand interactions. However, challenges arise in detection due to objects adjacent to the board and hands covering parts of the board. To overcome these challenges, we implement a detection pipeline that includes edge enhancement, contour filtering, polygon approximation, and tracking mechanisms across frames to ensure stable board localization.

Assumptions and Limitations

- Our core assumption is that the board is the largest quadrilateral object in the frame.
- The entire board should be visible for the first detection.
- Well defined edges are required for accurate detection, meaning blurred or low-contrast boards might be difficult to detect.

Implementation

The board detection process consists of the following steps:

1. Preprocessing

Before detecting the board, the image undergoes several preprocessing steps to enhance edge clarity and reduce noise:

- **Convert to Grayscale** – Simplifies the image by retaining only intensity values, setting up for edge detection.
- **Median Blur** (Kernel = 3×3) – Reduce noise while preserving strong edges, setting up for canny edge detection.
- **Canny Edge Detection** (Thresholds: 50, 150) – Retain significant edges, making the board's boundaries prominent.
- **Dilation** (Kernel = 5×5 , 2 Iterations) – First method to connect broken segments of the board's contour by expanding detected edges.

- **Morphological Closing** (Kernel = 5×5 , 3 Iterations) – Second method to connect broken segments of the board's contour by performing dilation & erosion such that the board's contour will close yet not expand the edges too much that they will cover the table as well.

By the end of this stage, the board's edges are clearly defined and ready for contour extraction.

2. Identifying the Board by Contour

Once the edges are well defined, the system searches for the largest quadrilateral contour that corresponds to the board:

- **Find Contours** – Extracts all shape boundaries from the processed edge image.
- **Sort Contours by Area** – Ensures the largest detected shape is prioritized, reducing the chance of false positives.
- **Polygon Approximation** – Uses the Douglas-Peucker algorithm to simplify detected shapes and remove noise.
- **Select the Largest Quadrilateral** – The first contour with exactly four corners is selected as the board.

3. Identifying the Board by Corners

If the board's shape was broken due to the adjacent objects or player's hands interference, we detect the board by the following method:

- Extract patches of the corners from the last frame where we successfully detected the board by the contour (meaning all 4 corners were visible) and extract patches of the same coordinates from the current frame.
- Compare the matching patches by **SSIM** score since it is variant to rotation but invariant to structure and lighting hence if the board didn't move the SSIM score will be high for each corner detected.
- If a corner received a SSIM score above 0.5, then it is considered as detected. If at least 2 corners were detected, logically the board didn't move, and we can extract it by the saved coordinates from the last successful frame.

With this approach, the system maintains stable board tracking even in cases of partial visibility.

4. Tracking the Board for Stability

Sometimes The board detection by contour algorithm detects corners with a slight offset from previous corners detected. We incorporate a tracking mechanism to maintain stability across frames:

- We calculate the distance between matching corners from the current frame and the last valid corners detected.
- If at least 2 corners moved a distance higher than a threshold of 5 pixels then the board moved and the offset wasn't noise, rather it's the movement itself.

With this approach, the system maintains stable board tracking even in cases of partial visibility or minor perspective changes.

5. Transforming the Board for our ROI

After detecting the board in the frame, we can extract it and transform it using.

Once the board's corners are detected and ordered, we perform a [perspective transformation](#) to align the board into a top-down view:

- Order Points – Arrange the corners consistently: top-left, top-right, bottom-right, bottom-left.
- Compute Dimensions – Calculate maximum board width and height using Euclidean distance between corners.
- Define Destination Corners – Set target rectangle corners based on computed dimensions.
- Get Perspective Matrix – Use [cv2.getPerspectiveTransform](#) to compute the transformation matrix from the original to the aligned board.
- Apply Warp – Use [cv2.warpPerspective](#) to generate a new image of the board, transformed to a rectified top-down view for consistent grid and object detection.

Real-Time Adaptation

The algorithm used is fast enough for our camera FPS rate, hence no adaptation is needed.

Results

The following presents the results of our board detection process, showcasing each step of the image processing pipeline. Each image demonstrates a crucial transformation that contributes to the accurate

detection and isolation of the backgammon board. The explanations highlight the importance of each step and justify the techniques used.

Step 1: Original Image

The initial image contains full scene, including the backgammon board, surrounding objects, and lighting conditions. At this stage, the board is not yet isolated, and all elements are present.



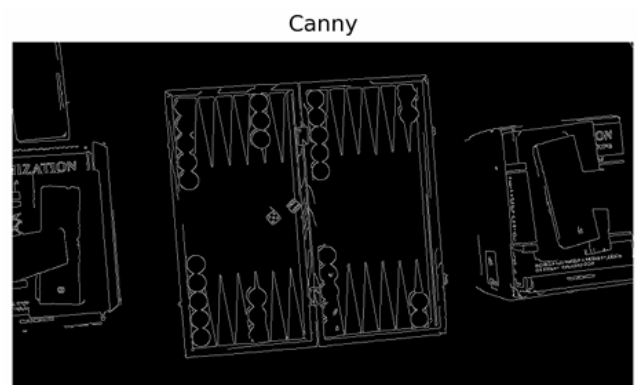
Step 2: Grayscale Conversion and Median Blur

The image is converted to grayscale and blurred using a median filter, removing color information and simplifying the processing.



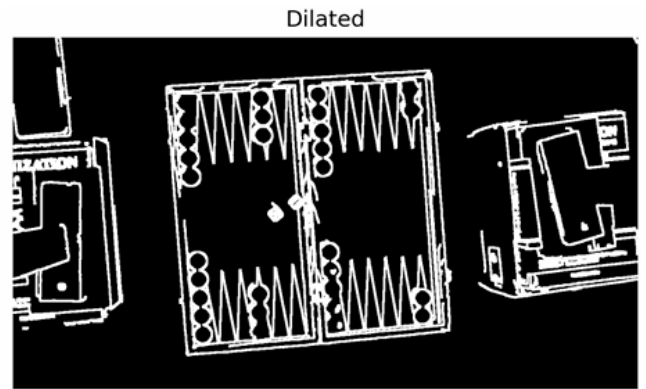
Step 3: Canny Edge Detection

The Canny edge detection algorithm is applied, highlighting the sharp intensity transitions in the image. At this stage, the board's edges become prominent, but smaller elements like checkers and dice also produce edges, which will be handled in later steps.



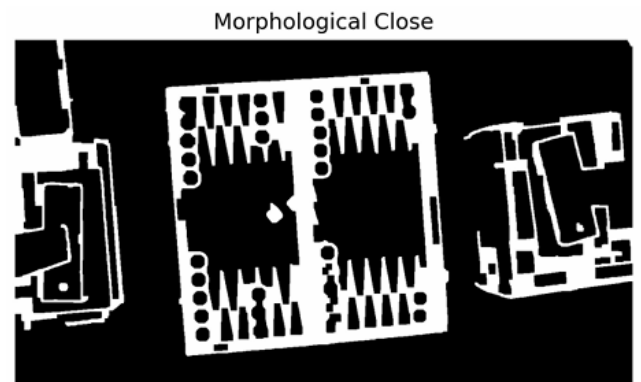
Step 4: Dilation

Dilation is performed to strengthen the detected edges, ensuring that broken contours are connected and small gaps in the board's outline are minimized. This step helps in forming a continuous border around the board, making it easier to identify as a single shape.



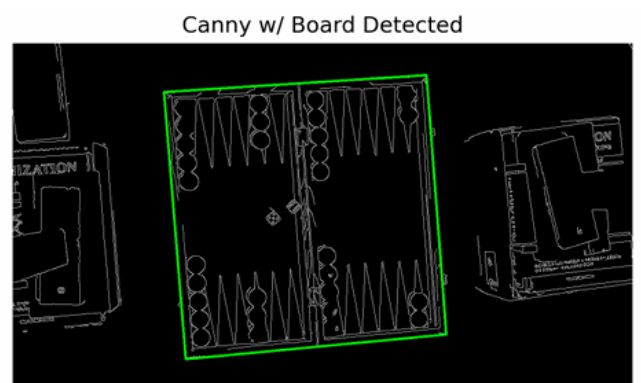
Step 5: Morphological Closing

A morphological closing operation is applied, filling in gaps inside the contours. This ensures that the board appears as a solid, enclosed region, making it easier to extract as a whole shape in the next steps.



Step 6: Board Contour Detection

Using contour analysis, the largest quadrilateral in the image is identified as the board, and a green bounding box is drawn around it. This step ensures that only the board is selected, excluding unnecessary elements.



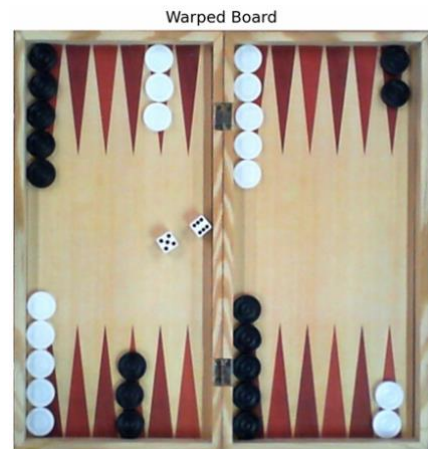
Step 7: Board Detected on Original Image

The detected board is now highlighted on the original image, confirming that the detection was successful and accurately follows the board's real-world positioning.



Step 8: Warped Board

The detected board is transformed into a top-down view, correcting any perspective distortions. This step ensures that all grid-based operations, such as checker and dice placement, remain accurate and aligned with the board's true dimensions.



Grid Creation

General Information

The grid creation algorithm defines the playable zones on the board by detecting 24 triangles that represent the game tiles. This structured grid is essential for tracking moves, detecting interactions, and maintaining an accurate game state representation, including the central bar tile.

To handle challenges such as lighting variations and weak triangle edges, we developed a method combining edge detection, contour filtering, shape validation, and geometric calculations to ensure precise and reliable triangle segmentation.

Assumptions and Limitations

- The algorithm assumes the board is empty during grid calculation; otherwise, checkers might interfere with triangle detection.
- It assumes the board was already detected and aligned to our desired perspective hence there are 24 visible triangles of same size, and the image contains only the board and its frame.

Implementation

1. Preprocessing

To enhance the board's features and improve triangle detection, we applied the following preprocessing steps:

- **Convert to Grayscale:** Removed color information to reduce computational complexity and improve contrast for edge detection.
- **Canny Edge Detection:** Identified intensity changes to highlight the triangular board elements. We used thresholds (20,100) instead of (50,150) used in board detection because triangle edges are more subtle, and we wanted to preserve contour continuity.
- **Dilation:** Strengthened edges and connected split lines to ensure fully formed triangles. We applied a 3×3 kernel with one iteration to avoid merging adjacent triangles.
- **Morphological Closing:** Filled small gaps inside triangle contours to make their shapes more complete. We applied three iterations to ensure smooth contours and prevent small breaks in the edges.

2. Detecting Triangles

After preprocessing, we identified all 24 board triangles using contour detection:

- **Find Contours**: Extracted shape boundaries to identify triangles. We used a retrieval mode that captures both inner and outer triangle edges to ensure no triangles were missed.
- **Approximating Triangle Shapes**: Since some contours contained extra vertices due to noise, we approximated the shapes to keep only their relevant corner points. We applied $\epsilon = 0.02 \times \text{perimeter}$, preventing excessive simplification while ensuring clear triangle boundaries.
- **First Filtering Step - Shape and Size Check**
 - We ensured that each triangle had at most 4 vertices.
 - We filtered out shapes smaller than a triangle's area (we calculated its area as a ratio from the board's size, since it's a fixed measurement across all frames) to remove noise from reflections or board imperfections.
- **Second Filtering Step - Compactness Check**
 - Triangles have a unique compactness value compared to irregular shapes.
 - We used the formula: $\text{Compactness} = \frac{4\pi \times \text{Area}}{\text{Perimeter}^2}$
 - Most valid triangles have a similar compactness, so we retained only triangles within 10% of the median compactness value.
- **Removing Duplicate Triangles**

Since we detected both inner and outer contours, we received duplicate triangle contours. We calculated the centroid of each triangle, and if two centroids were closer than the triangles base width, we discarded one of them.

3. Defining Grid Tile Boundaries

Once all 24 triangles were identified, we used them to define the tile boundaries:

- **Triangle Tile Calculation**:
 - Extracted the leftmost and rightmost points of each triangle to define its boundaries.
 - Calculated midpoints between adjacent triangles to create tile separation lines.

- Bar Tile Calculation:
 - The bar tile is built from the inner frame of the board.
 - To determine its width, we measured the board's frame width as the distance from the bases of the triangles and the start/end of the board image and defined the bar's width as twice the frame width.

4. Constructing the Grid Matrix

After defining tile boundaries, we created a structured grid matrix:

- We initialized a 2D matrix representation of the board, where each tile was assigned a unique identifier.
- Each tile was labeled based on standard backgammon numbering, allowing us to track checker positions accurately.
- Midpoints between detected triangles were used to define tile divisions precisely.
- The bar tile was assigned a separate label to ensure its integration within the grid system.

Real-Time Adaptation

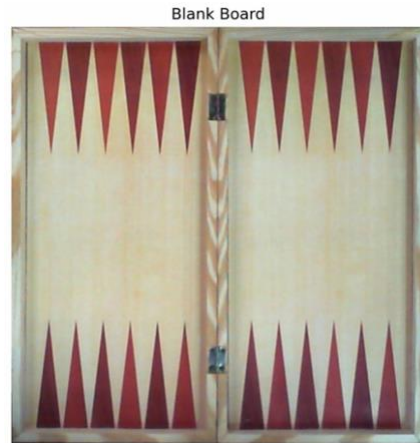
- The grid is computed only once at the start of the game, rather than recalculating it every frame, saving computational resources.

Results

The following images illustrate the process of grid creation, transforming the backgammon board into a structured, well-defined grid for game tracking. Each step refines the detection and segmentation of the triangular tiles, ensuring precise game state representation. Below is a breakdown of the results at each stage, explaining the role of each step in the overall process.

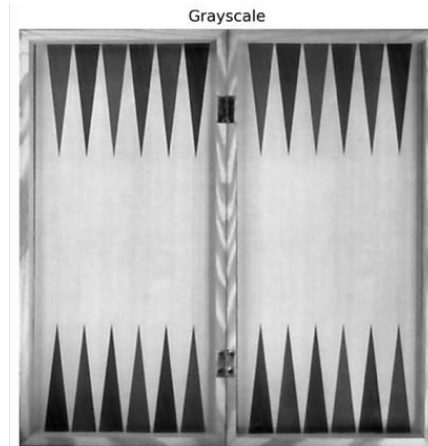
Step 1: Blank Board

At the start, we work with a clear image of the backgammon board without any checkers or dice. This ensures that grid detection focuses solely on the board's structure, without interference from game elements.



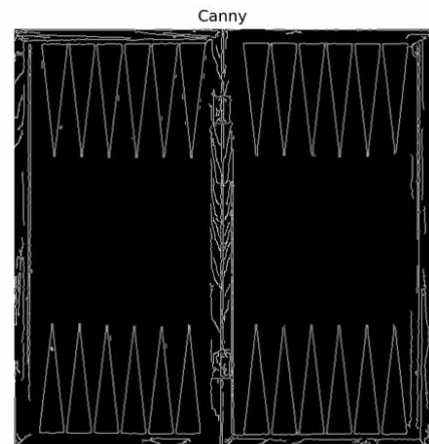
Step 2: Grayscale Conversion

The board is converted to grayscale, removing unnecessary color information. This enhances contrast between the triangular tiles and the background, making edge detection more reliable.



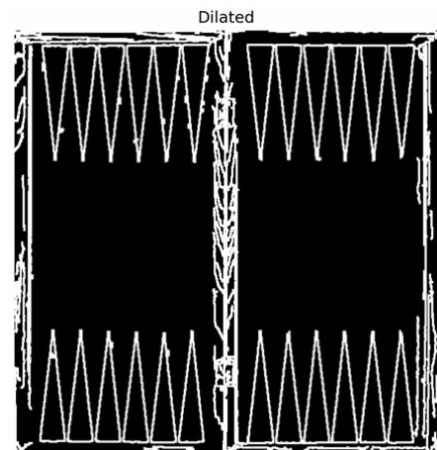
Step 3: Canny Edge Detection

Canny edge detection is applied to highlight the edges of the triangular tiles. By using threshold values optimized for subtle edges, we capture the full structure of the board while avoiding unwanted noise.



Step 4: Dilation

To ensure all triangle edges remain connected, we apply a dilation operation. This strengthens weak edges and merges fragmented contours while avoiding excessive merging of adjacent triangles.



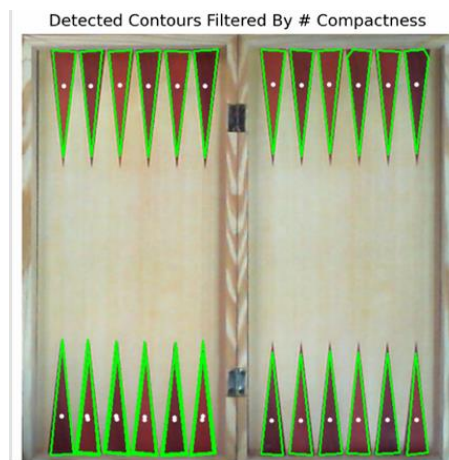
Step 5: Detecting Triangles by Corner Count

Contours are detected, and only those with a maximum of four vertices are retained. This filters out irregular shapes and ensures that only the triangular board tiles are considered in further processing.



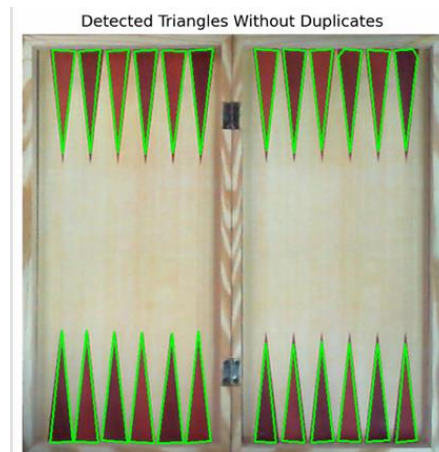
Step 6: Compactness-Based Filtering

To refine the selection, we apply a compactness check. Triangles have a specific compactness value, so we eliminate outliers that deviate too far from the expected shape characteristics.



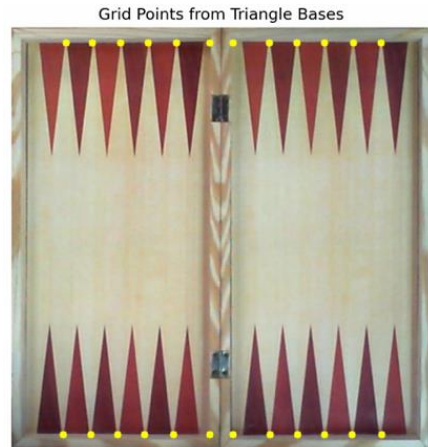
Step 7: Removing Duplicate Triangles

Since contours are detected on both inner and outer edges, duplicate detections can occur. To resolve this, we calculate the centroid of each triangle and discard those that are too close to another detected triangle.



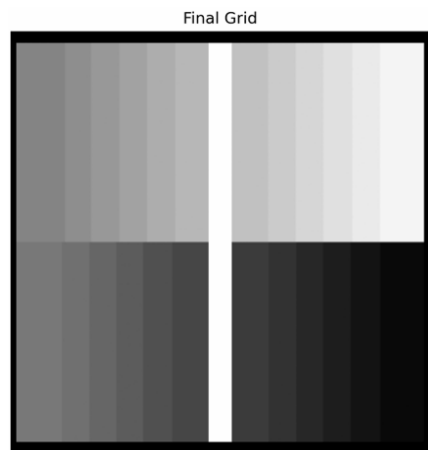
Step 8: Defining Grid Points

Once the correct triangles are identified, we extract the leftmost and rightmost points from each contour. These are used to define the grid structure, ensuring proper tile boundaries.



Step 9: Final Grid Representation

The board is divided into a structured grid, where each tile is assigned a unique index. This enables precise move tracking and interaction detection, forming the foundation for game state updates.



Checkers Detection

General Information

Checkers' detection is crucial for tracking the game state. The checkers' positions are determined by detecting their center of mass and mapping them onto the game grid. This process is crucial for moving validation and state updates. Additionally, detecting and subtracting white checkers from the board helps isolate the dice, improving their detection accuracy.

While checkers have a distinctive circular shape and color contrast (black and white), real-world challenges such as lighting variations, reflections, and overlapping checkers complicate detection. Therefore, a robust approach was required to reliably detect all checkers in various game conditions.

Initially, we explored several approaches:

1. **Support Vector Machine (SVM):** Used by texture classification with HOG features but proved unreliable due to lighting variations affecting the perceived texture of checkers in different board positions.
2. **Hough Circle Transform:** Applied due to the circular nature of checkers but failed due to lighting inconsistencies blending black checkers with their shadows or blending white checkers with light reflections, and stacked checkers distorting the circularity of the checkers.
3. **Final Approach - Adaptive Thresholding and Color Segmentation:** This was the most reliable method, utilizing the distinct black-and-white colors of checkers to achieve consistent detection. Using the masks created by thresholding we remove the texture of the checkers that interrupted us in the SVM solution.

Assumptions and Limitations

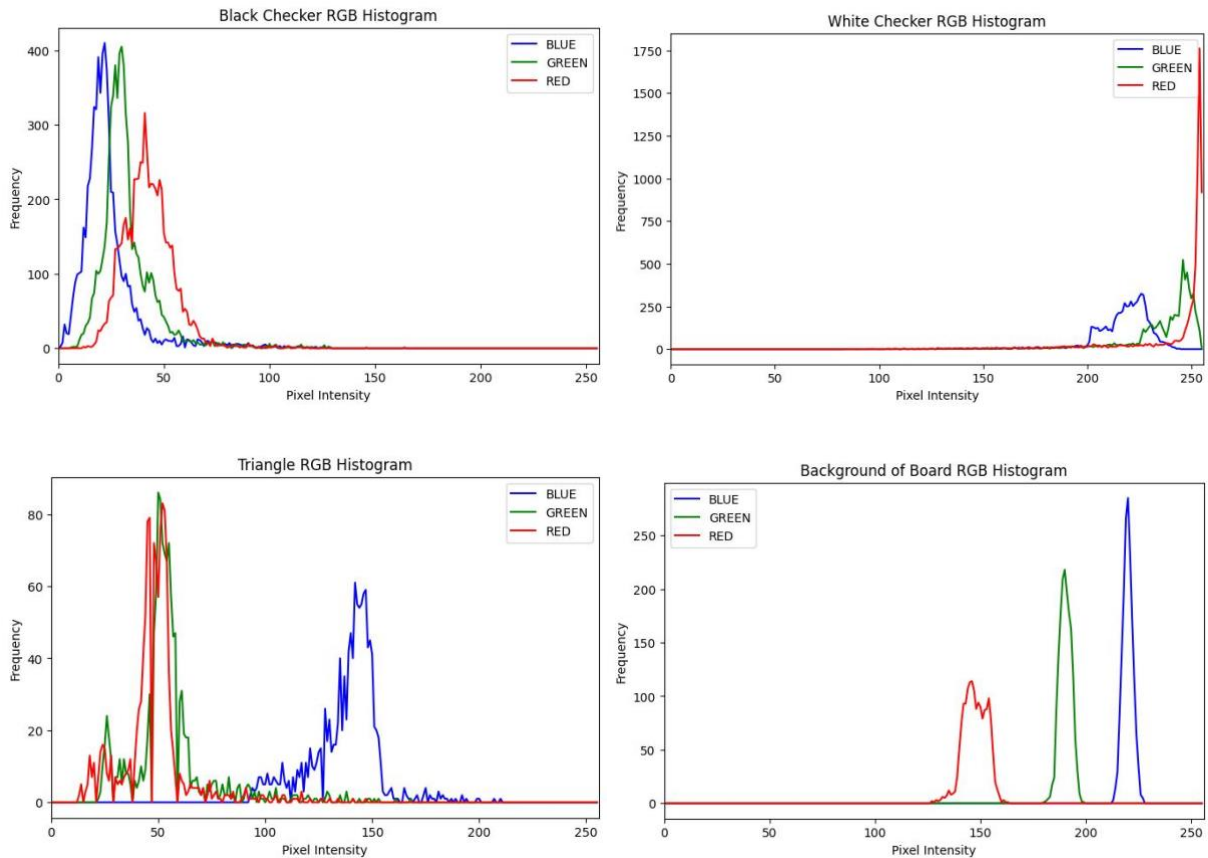
- **Checker Size Consistency:** Since the board detection algorithm aligns the board using projective transformation, we assume all checkers are perfect circles of uniform size.
- **Size Estimation:** The expected checker radius is derived from the board's detected dimensions, ensuring correct size filtering.
- **Lighting Sensitivity:** Although adaptive thresholding compensates for lighting variations, extreme reflections or shadows may still introduce minor inconsistencies.

Implementation

1. Checkers Segmentation – Adaptive Thresholding

Instead of using fixed thresholds that mis detect checkers, we analyze the pixel intensity histogram in each frame to dynamically distinguish checkers from the background.

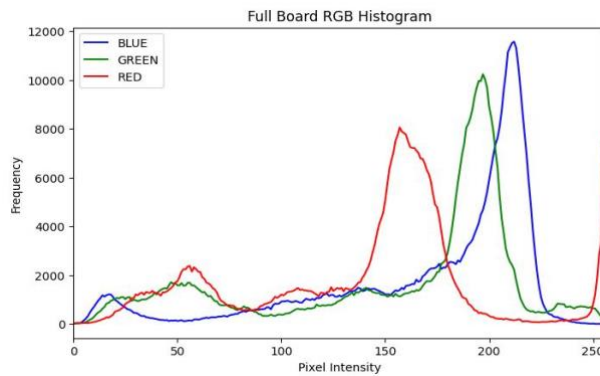
Analyzing the RGB histogram for each object in the game board we found a robust adaptive threshold to segment the checkers from the board:



We can observe that:

- Black checkers distribution has the lowest mean intensity in the blue channel with a narrow spread and they do not overlap with other objects (triangles or background) except for minor scattered noise.
- White checkers distribution has the highest mean intensity in the red channel with a narrow spread and they do not overlap with other objects (triangles or background) such that their intensity values remain distinct on the red channel.

Observing the histogram of the full board:

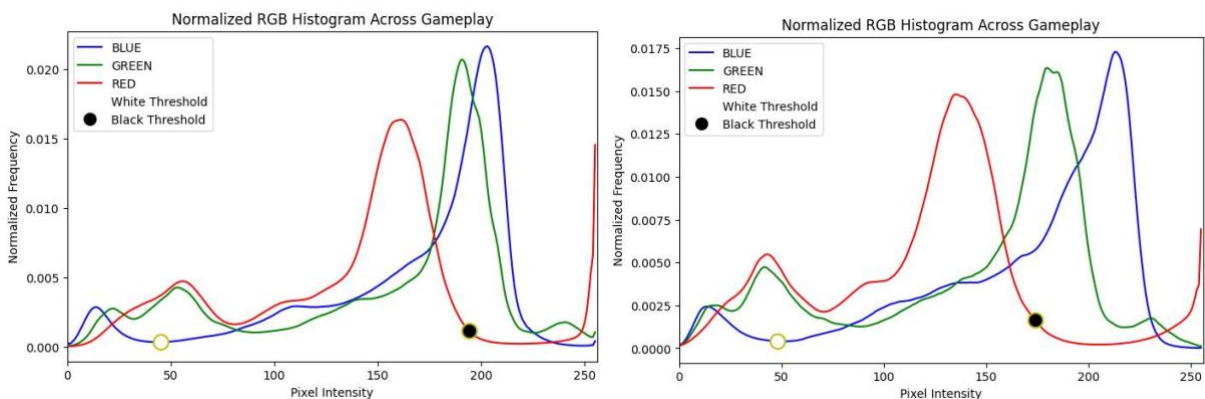


The first peak in the blue intensity histogram represents the values of the black checkers, while the last peak in the red intensity histogram represents the values of the white checkers. After analyzing multiple images with multiple positions of checkers, we found:

- The first minimum in the blue histogram separates the black checkers from the rest of the objects.
- The point where the largest peak discharges for 95% from the last minimum (a left offset from the last minimum) in the red histogram separates the white checkers from the rest of the objects.

We then applied two separate thresholding operations, creating binary masks. After inverting the black checkers' mask, we obtained two final binary images where checkers appear as white (255) against a black (0) background.

We can also see that the adaptive thresholding is needed since the histogram retain the same structure across frames but may shift left or right. Here's an example of averaging the histogram and calculating the mean of each threshold across 2 different gameplays:



2. Detecting Checkers' Centers of Mass

- **Template Matching:**
 - A circular template corresponding to the expected checker size is generated.
 - This template is matched across the masks to detect high-similarity points, representing checkers.
 - A similar threshold of 0.8 was chosen to ensure only strong matches close to the actual checker center are retained.
- **DBSCAN Clustering:**
 - Since template matching produces multiple detections for each checker, we apply clustering to group these detections into individual checkers.
 - Epsilon (eps) is set to the expected checker radius, ensuring that all detected points within a single checker remain in the same cluster while preventing merging of nearby checkers.
 - Minimum samples are set to 3, ensuring that even checkers with fewer detected points are retained while preventing false detections caused by reflections. while maintaining real-time speed for the algorithm (the larger the minimum samples, the slower the algorithm).
- **Computing the Center of Mass:**
 - Since template matching produces the strongest detections at the center of each checker and gradually decreases outward, averaging the clustered points helps find the center of the checker.
- **Detecting all the Checkers' pixels:**
 - Using a checker's center of mass and expected radius, we can identify all the checkers pixels by drawing a circle around the center.

Results

The following visualizations illustrate the key stages of the checker detection pipeline and demonstrate the effectiveness of each step in ensuring accurate and robust detection under real game conditions.

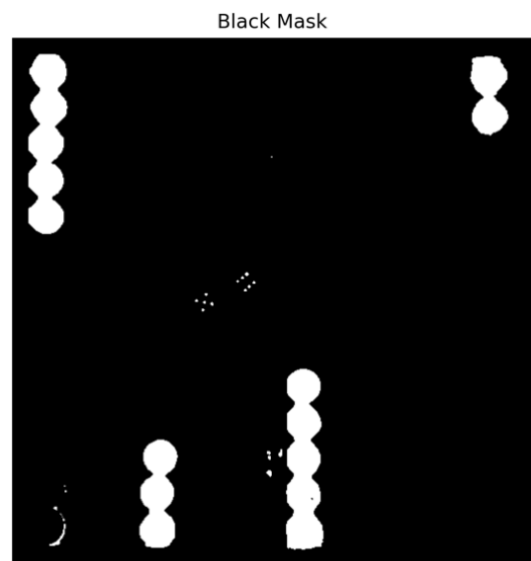
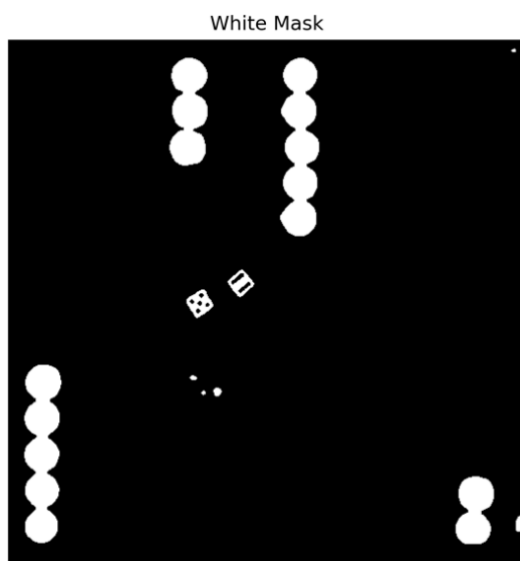
Step 1: Preprocessing:

using our board detection algorithm we cropped the board from the entire frame so there won't be pixels that could interrupt our histogram expected behavior.



Step 2: Checkers Segmentation

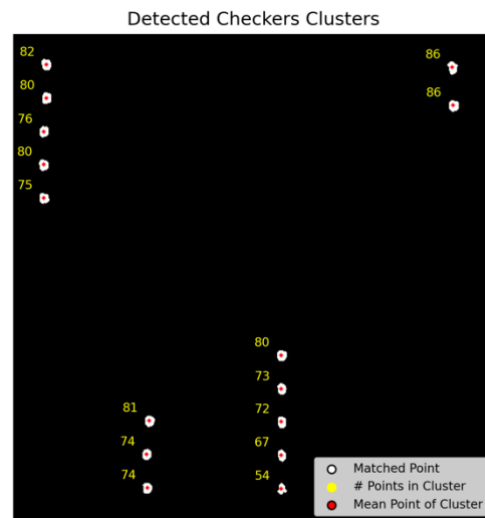
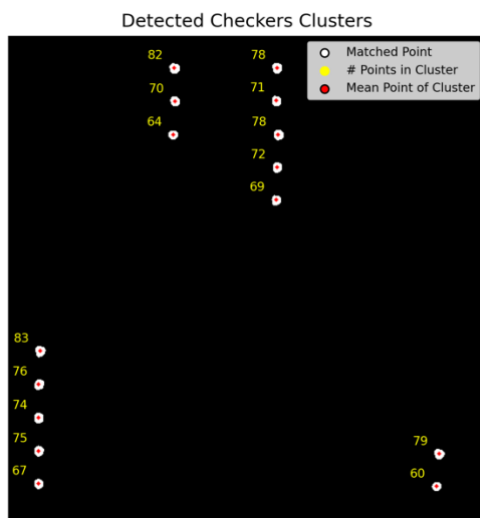
First, we found the adaptative thresholds using our algorithm and then we applied binary masks using these thresholds. In both masks, the checkers appear as distinct white blobs against a black background, clearly separated from the board and other elements like dice or shadows. This demonstrates the success of our segmentation method in cleanly isolating the checkers despite variations in lighting, reflections, or board texture.



Step 3: Detecting Checkers' Centers of Mass

Each white circle represents a detection from template matching, yellow numbers indicate the number of points in each cluster, and red dots mark the computed center of mass for each checker.

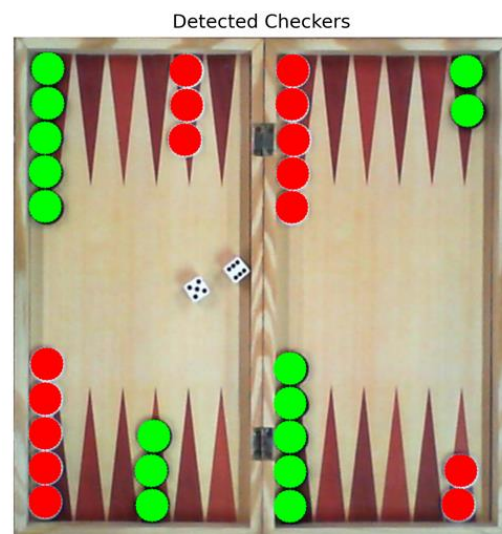
These results illustrate how overlapping detections were successfully grouped using DBSCAN clustering, and how the final positions were accurately extracted using the mean of each cluster. The absence of false clusters or noise points highlights the reliability of the combined matching and clustering process.



Finale Step: Detecting the Checkers

In this final stage, the segmented and clustered checkers are visualized directly on top of the original game frame. Each checker's detected center of mass is used to draw a filled circle with a fixed radius and color:

- Red circles represent black checkers
- Green circles represent white checkers



Dice Detection

General Information

Dice detection is essential for determining the current game state, as the outcome of the dice roll directly affects the player's move. Based on the assumption that the dice are white cubes with black dots, we isolate them using color filtering and shape analysis. However, since dice visually resembles white checkers, and the black dots disrupt the square shape of the dice, combined with the possibility that the dice may be touching each other, the detection process must be precise. To address these challenges, we employ a pipeline that includes preprocessing, contour filtering, rotation correction, intelligent cropping, and dot detection through template matching.

Assumptions and Limitations

- The algorithm assumes that dice are white cubes with black dots, having a fixed size compared to the board size, and located only within the board area.
- The dice must not be placed on top of checkers, otherwise the algorithm won't detect them and move to the next frame.
- When two dice are touching, they may appear as a single contour. The algorithm handles this only when the combined contour has an appropriate geometric ratio (e.g., 2:1).
- Dot detection assumes that each die contains between 1 and 6 visible black dots.
- We've tried to calculate the ratio between the white pixels (dots) to black pixels (dice) to determine the number of dots on a detected dice. This method failed since we received the same ratio and same number of white pixels for different numbers (e.g 4,5 and 2,3).

Implementation

1. Preprocessing

- **Subtracting white checkers** from the white mask. This operation ensures that only the dice remains in the processed mask.
- **Median blur** with a kernel size of 5 is applied to reduce noise and smooth inconsistencies caused by imperfect subtraction of the white checkers.
- **Dilation** operation is performed using a 3×3 kernel for one iteration. This reinforces the shape of the dice and reconnects any fragmented parts that may have resulted from the black dots on their surface.

2. Detecting Dice

- **Find Contours** on the preprocessed image.
- **Filter** contours by **area** to retain only contours of the size of the dice. Since the dice are of fixed size compared to board's size, this is possible.
- **Filter** contours by **aspect ratio** of each contour's minimum bounding box: for a single die, we expect a near-square ratio (between 0.8 and 1.2), while for two adjacent dice merged into one contour, we look for a rectangular shape (between 0.3 and 0.7).

3. Cropping and Aligning the Dice

- Once a die has been detected, we use `cv2.minAreaRect` to obtain its rotation angle and generate a rotation matrix to align the die vertically. This correction simplifies the next step of dot detection.
- After alignment, we extract the rotated die from the black mask using its new bounding box. If the shape represents two connected dice, we split it in half-either horizontally or vertically depending on its orientation.
- To eliminate edge noise that was prominent across multiple frames of false detections, we crop 5% from each side of the die image. This preserves the inner area containing the dots while removing distracting border noise that interferes with detection.

4. Dot Detection and Counting

- **Template Matching**: With a clean image of each die, we apply template matching using a circular kernel that mimics the shape of a die dot. The matching is performed on the binary die image to locate all potential dot positions.
- **DBSCAN Clustering**: The resulting match points are then clustered using the DBSCAN algorithm to receive the number of dots detected on the die.

Real-Time Adaptation

- After detecting a new dice throw (new numbers detected) we wait another 2 frames to let the dice set since our frame rate is too fast also capturing the dice during the throw.

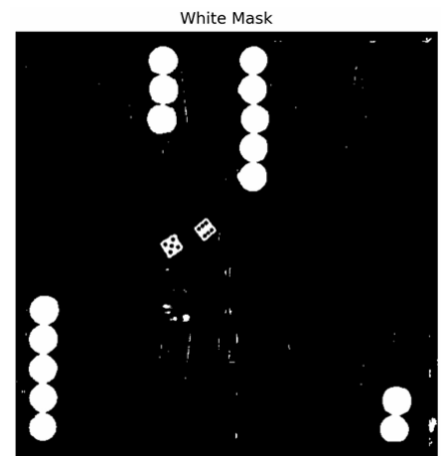
Results

The following images present the results of the dice detection process, from isolating the dice from checkers to accurately counting the number of dots. Each image represents a critical step in the algorithm, demonstrating how various image processing techniques refine and enhance detection accuracy. The developed method ensures high precision in real-time, even under challenging conditions such as varying lighting, adjacent dice to white checkers, and adjacent dice.

Below is a breakdown of the results at each stage, with a concise explanation of how each step contributes to the overall detection process.

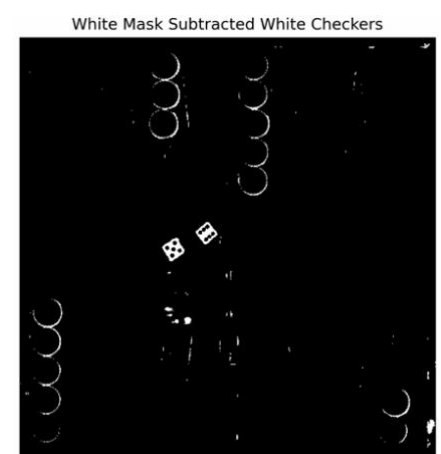
Step 1: White Mask

At this stage, all bright elements in the scene, including white checkers and dice, are highlighted in white on a black background. Since checkers and dice appear identical, additional filtering is required to differentiate them before focusing on dice detection.



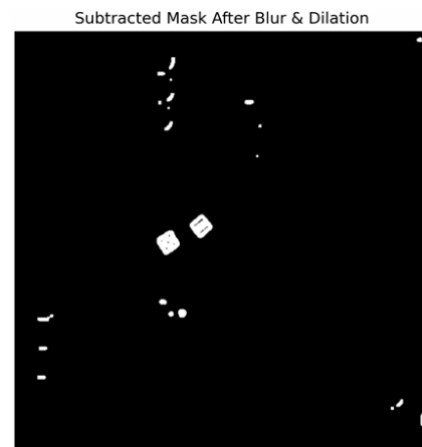
Step 2: White Mask with Subtracted Checkers

The white checkers have been removed from the mask, leaving only the dice and some residual noise. This ensures that the detection process focuses solely on the dice, preventing false identification caused by checkers.



Step 3: Blurred and Dilated Mask

A median blur and dilation operation are applied to enhance the dice shapes. Median blur smooths out imperfections and reduces noise caused by checker subtraction. Dilation reconnects fragmented edges of the dice, ensuring they remain as solid, well-defined objects for contour detection.



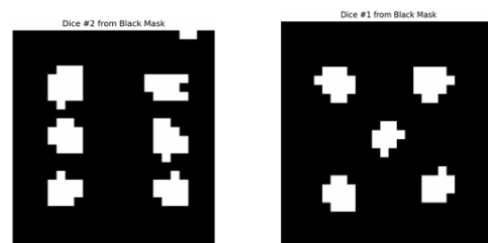
Step 4: Detected Dice

The dice contours have been successfully detected and highlighted in yellow bounding boxes. The algorithm filters objects based on size and shape, ensuring that only valid dice are retained. If two dice are touching, the system considers their combined contour and prepares them for separation.



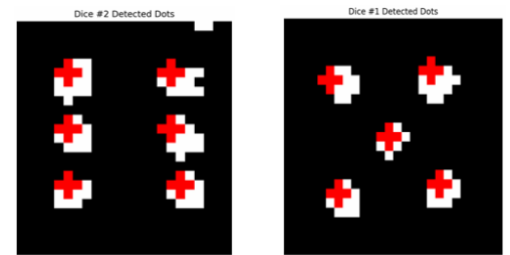
Step 5: Cropping and Isolating Individual Dice

The identified dice are cropped and isolated from the black mask, ensuring a clean background where only the dice remains. This prevents distractions and prepares them for dot detection by eliminating unnecessary elements.

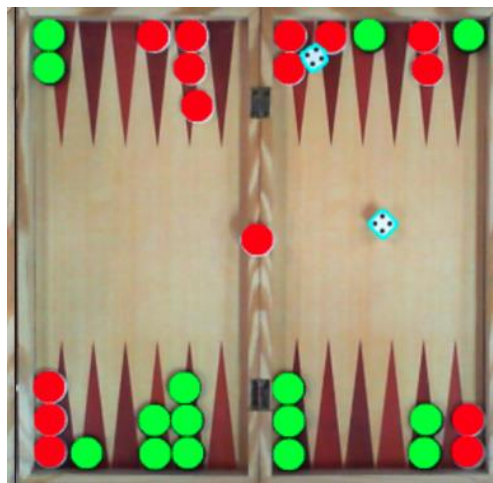


Step 6: Dot Detection on Dice

The detected dots on the first die are highlighted in red. Using template matching and DBSCAN clustering, the system accurately locates and groups the dots, ensuring a precise count for determining the die's value



Here is an example of detecting the dice although it being adjacent to 3 white checkers



Hand Detection

General Information

Detecting a player's hand is a critical component in understanding interactions with the dice, especially identifying when a roll is about to occur. We assume that a player reaches toward the dice right before rolling, and therefore hand detection helps the system recognize real-time actions. The main challenge is distinguishing between actual hand movement and background noise, lighting changes, or general movement in the scene. Our approach is based on calculating optical flow between a clean background frame (no hand) and a new frame (with hand), followed by contour analysis to locate the hand region.

Assumptions and Limitations

- The algorithm assumes access to a clean background frame with no hand present for comparison.
- Only one hand is expected to appear in the scene at any given time.
- The hand must be moving to be detected – static hands may not trigger motion detection.
- Sudden lighting changes may result in false motion detection.
- The hands skin color is like the background color of the board hence any color space segmentation enchantment doesn't work.
- The algorithm assumes that once a player takes the dice, he will throw it later.

Implementation

1. Motion-Based Preprocessing

The algorithm begins by resizing both the background and current frame to half their original size to improve real-time performance. The frames are then converted to grayscale to simplify calculations. Using Farneback's method, we compute dense optical flow between the frames, which provides a motion vector (magnitude and angle) for every pixel. We normalize the magnitude to a range of 0–255 and apply a threshold to extract only regions of meaningful motion-assumed to represent the moving hand.

2. Contour Detection and Hand Localization

Contours are extracted from the threshold motion mask. If any contours are detected, we assume the largest one corresponds to the hand. From this contour, we extract both a minimum area rectangle (via

cv2.minAreaRect) and a standard bounding box (x, y, w, h). To prevent misclassification, we only accept hand candidates with a bounding box area smaller than $\frac{3}{4}$ of the board's size – this filters out large irrelevant motion created by light flickering. We also track previous hand positions to compute movement direction and ensure continuity.

3. Detecting Cover and Release Events

To detect whether the hand covers the dice, we check whether all dice centers from the previous frame fall within the hand's bounding box. If they do, we consider the dice as “covered”. Once the hand moves away and the dice disappear (dice count becomes zero), we mark it as a “taken” event. At this moment, the system determines which player took the dice by checking whether the hand's vertical position is above or below the board's midpoint. This determines who will throw the dice next.

Real-Time Adaptation

- Both frames are resized by 50% before processing to reduce computation time.
- Dense optical flow (Farneback as opposed to Lucas-Kanade) provides per-pixel motion analysis, enabling accurate movement localization

Results

The following images illustrate the results of the hand detection process, highlighting how the algorithm isolates hand motion and accurately identifies its position over the board. The pipeline ensures real-time responsiveness, detecting hand presence and movement with high precision while filtering out irrelevant motion. Below is a breakdown of the results at each stage, with explanations of how each step contributes to the overall detection process.

Step 1: Motion Mask

This image represents the motion mask, where red areas indicate detected movement.

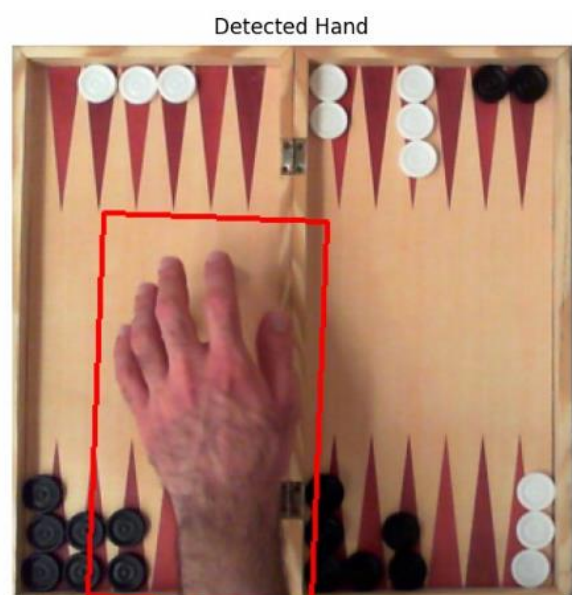
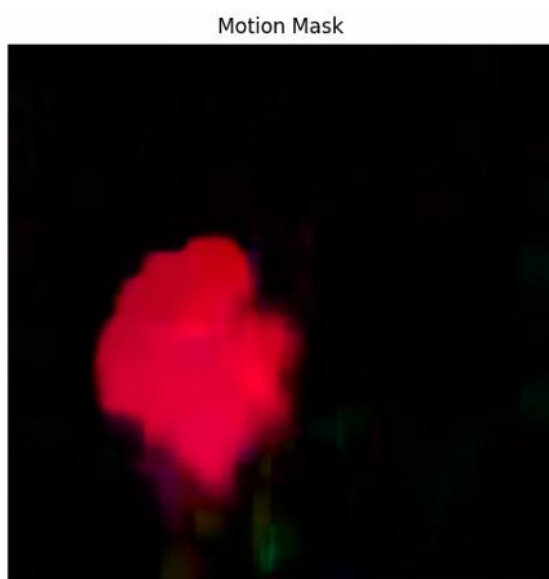
- The optical flow algorithm tracks pixel displacement between frames, allowing the system to focus only on regions where motion is occurring.
- The hand appears clearly in red, while the rest of the board remains dark, confirming that motion detection is correctly isolating moving objects.

Step 2: Detected Hand with Bounding Box

In this image, the detected hand is enclosed within a red bounding box, marking the region identified as the hand.

- The contour extraction process detects the largest moving object in the motion mask, assuming it represents the hand.
- The bounding box ensures that the detected area remains stable across frames, improving tracking reliability.
- The box's dimensions help filter out unwanted motion (such as shadows or small object shifts) by applying a size threshold.

At this stage, the system has successfully localized the hand and is ready to analyze its interaction with the dice.



Algorithms Analysis

In this section, we present the main algorithms we used throughout the project, focusing on the theoretical concepts behind them. For each algorithm, we explain the logic behind it, how it relates to topics covered in the course, and specify its input, output, key parameters, and the assumptions required for proper implementation.

Gaussian and Median Blur

Both methods are convolving a LPF with the image to smooth it or remove noise as learnt in Lecture 2 about image filtering and implemented by us from scratch in the first home assignment.

Gaussian Blur

Gaussian Blur is primarily used to remove Gaussian noise or to smooth an image by applying a weighted average, where nearby pixels contribute more than distant ones according to a Gaussian distribution. It is important to note that the edges are smoothed out, and Gaussian filtering introduces new pixel values into the image.

The filter values are calculated using the 2D Gaussian distribution formula:

$$G(x, y) = \frac{1}{2\pi\sigma_x^2\sigma_y^2} e^{-\left(\frac{x^2}{2\sigma_x^2} + \frac{y^2}{2\sigma_y^2}\right)}$$

x and y represent pixel coordinates within the filter, and σ determines the level of blurring.

Input:

- **Image** – The source image to convolve with the filter. There are no restrictions to the input.
- **Kernel Size** – The size of the kernel of the filter (must be an odd number). The larger the filter, the more computationally expensive the method is.

Output:

- **Image** – Filtered source image. Might contain new pixel values due to the filtering.

Median Blur

Median Blur replaces each pixel in the image with the median value of its surrounding $k \times k$ neighbors. This technique is especially effective in removing impulse noise (also known as salt-and-pepper noise) while preserving image edges. Unlike the Gaussian filter, which can be influenced by extreme pixel values, the median filter is more robust to outliers.

The pixel transformation is defined as:

$$I'(x,y) = \text{median}\{I(i,j) \mid (i,j) \in \text{neighbor}(x,y)\}$$

This means that for each pixel $I(x,y)$, the median of all neighboring pixel values is selected, replacing the original pixel value.

Input:

- **Image** – Same as in Gaussian Blur.
- **Kernel Size** – Same as in Gaussian Blur.

Output:

- **Image** – Filtered source image.

Canny Edge Detection

Canny edge detection identifies sharp intensity changes in an image to detect edges and boundaries of objects. We've learned this algorithm in Lecture 7 and analyzed in home assignment 4.

Input:

- **Image** – Grayscale image, since edge detection relies on intensity variations between adjacent pixels.
- **Low Threshold** – Used for hysteresis threshold.
- **High Threshold** – Used for hysteresis threshold.

Output:

- **Image** – Binary image, where white pixels (255) represent detected edges, and black pixels (0) represent non-edge areas.

Algorithm:

1. Filter image with x, y derivatives of gaussian:

Reduces noise by applying a gaussian filter and calculating the derivative at each pixel. Noise reduction is necessary to prevent false edge detections.

2. Compute magnitude and orientation of gradient:

Sharp intensity transitions are detected using the Sobel operator by computing the gradient magnitude and direction:

$$G = \sqrt{G_x^2 + G_y^2}, \theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

3. Non-Maximum Suppression:

Only the highest magnitude valued pixels in the gradient direction are retained to produce sharp, thin edges:

$$G(x, y) = \begin{cases} G(x, y), & \text{if } G(x, y) \text{ is the local maximum in the gradient direction} \\ 0, & \text{otherwise} \end{cases}$$

4. Hysteresis Thresholding:

- a. Define two thresholds – high and low.
- b. Pixels with magnitude higher than the high threshold are considered “strong edges”, pixels with magnitude between low and high threshold are “weak edges” and pixels lower magnitude than the low thresholds are considered as noise and are removed.

- c. “Strong edges” are retained, while “weak edges” are preserved only if they are connected to strong edges. This step ensures that only meaningful and continuous edges remain, while isolated weak edges are discarded.

Open-CV’s Implementation

1. Gaussian filter for noise reduction is not performed in the open-cv’s canny edge detection function, hence it is needed to perform beforehand by us.
2. Gradient calculation has a default Sobel filter of kernel size = 3.
3. The gradient magnitude calculation is calculated at default as the L1 norm. can be switched to L2 by a flag.

Dilation

Dilation is an algorithm that expands bright regions in an image, particularly useful in our case for connecting disjointed features, filling small gaps and closing contours of shapes.

The transformation is given by:

$$I'(x, y) = \max_{(i,j) \in \text{kernel}} I(x + i, y + j)$$

Each pixel is replaced by the maximum value in its neighborhood, as defined by the kernel.

Input:

- **Image** – Binary image with target features as white (higher value).
- **Kernel** – The kernel used for the algorithm.
- **Iterations** – Number of iterations to perform dilation.

Output:

- **Image** – binary image.

Algorithm:

In a binary image (as we only use it), We convolve the kernel over all the pixels in the image and the pixel value will be the maximum value of all neighboring pixels covered by the kernel, hence in a binary image, a pixel element is '1' if at least one pixel under the kernel is '1'. So, it increases the white region in the image or size of foreground object increases.

Erosion

Erosion operates similarly to dilation, but instead of expanding bright regions, it shrinks them. The input and output of the function is the same, the only difference is while dilation replaces a pixel with the maximum value of its neighbors under the kernel, erosion replaces it with the minimum value.

Mathematically, the operation is defined as:

$$I'(x, y) = \min_{(i,j) \in \text{kernel}} I(x + i, y + j)$$

In other words, each pixel is assigned the lowest value among the pixels covered by the kernel. Therefore, in a binary image, after erosion, a pixel will be '1' only if all the pixels under the kernel are also '1'. This process causes bright areas to contract, making objects thinner, disconnecting close elements, and removing small bright noise.

Morphological Operation – Closing

Closing is a morphological operation that performs dilation followed by erosion on a given image. This process enables the merging of broken components and improves the continuity of objects in the image.

The operation takes as input a grayscale or binary image, along with a kernel that defines the region where the dilation and erosion are applied. Additionally, a number of iterations can be specified, which affects the extent of the operation such that more iterations result in a stronger closing effect.

The output is an image where small gaps within bright regions have been closed, making previously broken or disconnected objects more continuous. This enhances the ability to identify complete shapes and improves image quality for further processing.

The operation depends on two key parameters:

- **Kernel** – Defines the size of the closing effect, determining the area where pixels will be expanded and contracted.
- **Number of Iterations** – Controls the intensity of the operation, where a higher value results in more significant closing of bright regions.

The closing operation is mathematically defined as:

$$I'(x, y) = \min_{(i, j) \in \text{kernel}} \left(\max_{(i, j) \in \text{kernel}} I(x + i, y + j) \right)$$

(i, j) represent the coordinates of the structuring element relative to its center.

The algorithm assumes that objects of interest are bright, while the background is dark, meaning that small dark gaps within bright objects are considered noise and should be filled to maintain the integrity of their shape. Additionally, it assumes that closely positioned bright regions should be connected, making closing an effective way to improve the continuity of such areas.

Find Contours (Contour Detection)

FindContours is an open-cv's function used to detect the boundaries of objects in an image. It identifies connected edges in a binary image and represents them as a list of lines or points.

Input:

- **Image** – Binary image where the object edges are white, and the background is black.
- **Retrieval Mode** – The type of contours to detect. We used the following methods:
 - `cv2.RETR_EXTERNAL` – Detects only the outer contours of objects.
 - `cv2.RETR_TREE` – Detects both outer and inner contours of objects.
- **Approximation Method** – We always used the `cv2.CHAIN_APPROX_SIMPLE` method which saves a contour as the essential points constructing it (meaning if a segment of the contour is a straight line, it saves only the endpoints of that segment since we can reconstruct the line by those 2 points only) instead of saving all the points constructing the contour. This will help us later when we approximate the polygon of the contour.

Output:

- **List of contours** – Each detected object is represented as an array of contour points, defining its boundary.
- **Hierarchy** – If hierarchical relationships exist between contours (e.g., an inner contour inside a larger shape), this information is stored. We did not use this in our project.

Algorithm:

It uses Suzuki and Abe's algorithm, known as the Border Following algorithm as follows:

1. Scans the image pixel-wise from the top-left corner.
2. When it finds a white pixel without a label:
 - a. If it's on the edge of a new object, it's an external contour.
 - b. If it's on the inside of a black region within a white object, it's an internal contour.
3. For each such pixel, the algorithm:
 - a. Traces the border of the object in clockwise (external) or counterclockwise (internal) direction.
 - b. Records the coordinates of each boundary pixel.
 - c. Assigns the same label to connected border pixel.

Approximate Polygon

Approximates a given polygon with another polygon with less vertices so that the perimeter between them is less or equal to the specified precision.

Input:

- **Polygon Points** – List of the points defining the polygon.
- **Epsilon** – The approximation accuracy. This is the maximum distance between the original curve and its approximation used in Douglas-Peucker algorithm.
- **Closed Flag** – Define whether the given points describe a polygon or a curve hence if the first and last point must connect. We always set it as “True” since we always approximated polygons and not curves.

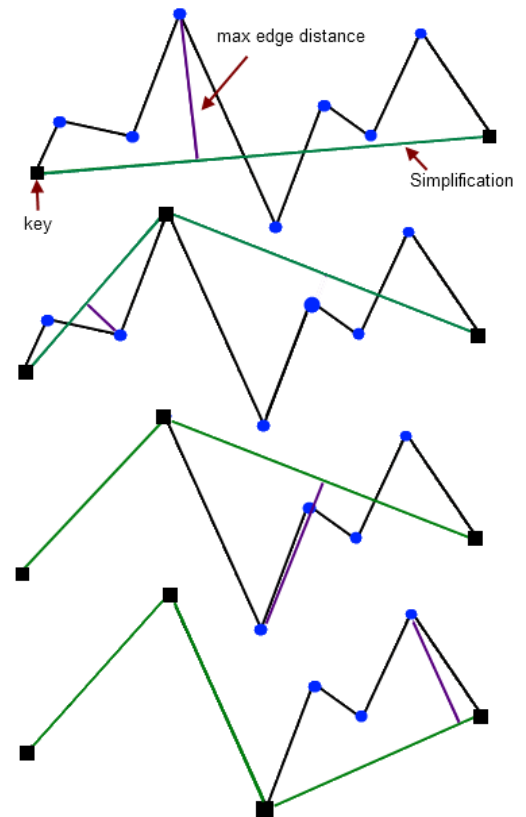
Output:

- **Polygon Points** – List of the points defining the approximated polygon.

Algorithm:

It uses Douglas-Peucker algorithm as follows:

1. Connect the first point P_0 and the last point P_n with a straight line.
2. Find the point P_{max} that is the farthest from this line segment. Measure perpendicular distance from each intermediate point to the line.
3. Check distance:
 - If the max distance $>$ epsilon, keep P_{max} as a significant point.
 - Recursively apply the algorithm to the segments P_0 to P_{max} , and P_{max} to P_n .
 - If the max distance $\leq$ epsilon, then all intermediate points are removed (line is a good enough approximation).
4. Merge Results: Combine the points that survived all recursive steps to form the simplified curve.



Template Matching

Template Matching is an image filtering technique used to find a smaller template image within a larger image. The method works by sliding the template over the source image and computing a similarity score at each position. The best match is determined based on the highest (or lowest) score, depending on the selected matching method.

Input:

- **Source Image** – The image to find the template.
- **Template Image** – The template image to slide over the source image.
- **Matching Method** – Define the similarity score between the template and different regions of the source image.

Output:

- **2D Matrix** – Result matrix, where each value represents the matching score at the corresponding position (top-left corner of the detected region).

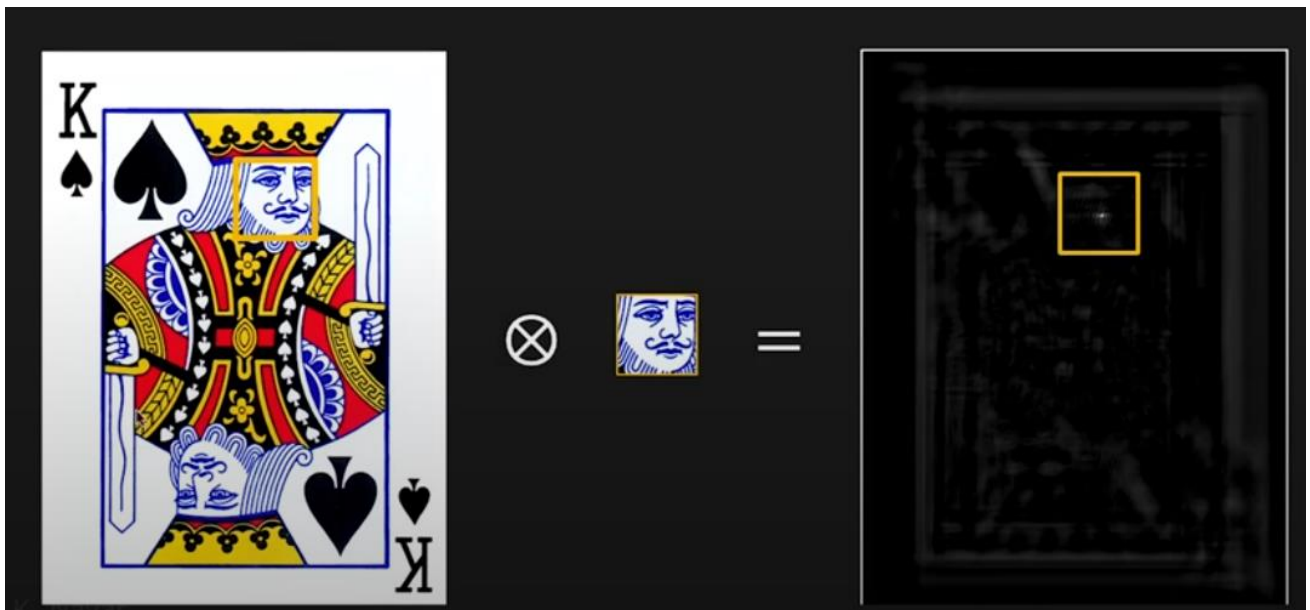
Algorithm:

We chose to use cv2.TM_CCORR_NORMED matching method, which is based on normalized cross-correlation and provides higher values for better matches.

Mathematical Formula:

$$R(x, y) = \frac{\sum_{i,j} I(x + i, y + j) \cdot T(i, j)}{\sqrt{\sum_{i,j} I(x + i, y + j)^2 \cdot \sum_{i,j} T(i, j)^2}}$$

$I(x, y)$ is the source image, $T(i, j)$ is the template, and the denominator normalizes the result, ensuring that values remain within the range $[0,1]$, making the method less sensitive to changes in lighting conditions. The Template Matching algorithm assumes that the template is identical or nearly identical to a region in the source image, meaning significant variations in appearance may lead to inaccurate detections. It also assumes that the size and rotation of the template remain unchanged, so if the template appears in the image at a different scale or orientation, it will not be detected correctly. Additionally, the algorithm relies on consistent lighting conditions, as changes in brightness can affect similarity scores, unless a method that compensates for illumination differences, such as TM_CCOEFF, is used. Since we use it to detect circular (rotation invariant) objects (checkers and dice dots) in a binary image we overcome its constraints.



This image demonstrates how the Template Matching algorithm works for pattern localization-in this case, detecting the king's face from a playing card. On the left is the original image, in the center is the template we aim to locate, and on the right is the resulting heatmap generated by sliding the template across the image. Each pixel in the heatmap represents the similarity between the template and a specific region in the image-higher values indicate better matches. In this example, the similarity metric used is `cv2.TM_CCORR_NORMED`, which calculates the normalized cross-correlation between the template and each image patch. This method compensates for lighting differences and pixel intensity, producing values between 0 and 1, where higher values indicate stronger correlation. the white point (highlighted in yellow) marks the location with the highest match score-meaning the best match between the template and the image was found there.

Affine & Projective Transforms

Affine and projective transforms are fundamental geometric operations in computer vision, used to align, warp, and correct images. In our project, we applied these transformations to detect and rectify the backgammon board from a camera frame, converting a skewed or angled view into a clean, top-down perspective. This process is essential for ensuring consistent grid construction and accurate game state analysis. These topics were introduced in Lecture 9 (Geometric Transformations) and implemented in the detect board function.

Affine Transform

Affine transformation preserves collinearity, parallelism, and ratios between points. It allows image transformations such as translation, rotation, scaling, and shearing, but it does not account for perspective distortion.

Mathematical Definition:

$$I'(x, y) = A \cdot I(x, y) + b$$

The point $I(x, y)$ represents the original pixel location, A is a 2×2 transformation matrix that applies rotation, scaling, and shearing, and b is a translation vector that shifts the pixel to a new position.

Input:

- **Image** – Source image to transform.
- **Source Points** – 3 points from the original image.
- **Destination Points** – 3 corresponding points in the target configuration.

Output:

- **Transformed Image** – The output image after applying the affine transformation.

Projective Transform

Projective transformation maps points from one plane to another while accounting for perspective distortion, making it ideal when the image is viewed from an angle. It preserves straight lines but does not preserve parallelism.

Mathematical Definition:

$$I'(x, y) = H \cdot I(x, y)$$

The point $I(x, y)$ represents the original pixel location, H is a 3×3 homography matrix that applies translation, rotation, scaling, shearing, and perspective distortion, and $I'(x, y)$ is the new pixel location after transformation. This allows mapping between planes with perspective effects.

Input:

- **Image** – Source image with perspective distortion.
- **Source Points** - 4 corner points from the original image.
- **Destination Points** - 4 corresponding points in the desired aligned view.

Output:

- **Warped Image** – The perspective-corrected result.

We used OpenCV's built-in functions:

- `cv2.getAffineTransform(srcPoints, dstPoints)` – calculates the 2×3 matrix.
- `cv2.warpAffine(image, matrix, output_size)` – applies the transformation.
- `cv2.getPerspectiveTransform(srcPoints, dstPoints)` – calculates the 3×3 homography matrix.
- `cv2.warpPerspective(image, matrix, output_size)` – applies the transformation.

Bounding Box

A Bounding Box is a basic tool in object detection used to bound an object in an image. It defines a straight rectangular box that tightly encloses the object, allowing for further operations such as tracking, classification, cropping, or graphical display.

In our project, Bounding Boxes were used to calculate the ratio between edges of a polygon (board and dice) or to bound an object (hand).

Input:

- **Contour / Coordinates** – A list of contour points or pixels representing the object.

Output:

- **Bounding Box** – The smallest rectangle enclosing the object, typically in one of these formats:
 - (x, y, width, height)
 - (x1, y1), (x2, y2) (top-left and bottom-right corners)

Algorithm:

1. Receive a set of contour points (or coordinates).
2. Find the minimum and maximum x and y values.
3. Construct a rectangle using these values.
4. Optionally, expand the box with padding or aspect ratio adjustments.

We used OpenCV's built-in functions:

- `cv2.boundingRect(contour)` – Returns the smallest upright rectangle (x, y, w, h) that encloses a contour.
- `cv2.minAreaRect(contour)` – Returns a rotated rectangle with minimum area that fits the contour.

DBScan Clustering

In our project, we used DBSCAN after template matching to group overlapping checker detections. Since template matching often returns multiple close detections for a single checker, DBSCAN allowed us to cluster those detections and merge them into a single, averaged checker position. This step significantly improved the robustness of checker detection, especially when dealing with noisy or ambiguous template responses. It also filtered false positives resulting from reflections, strong edges, or adjacent checkers.

Input:

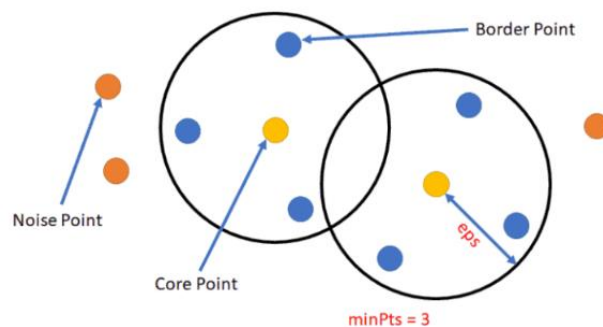
- **Raw coordinates:** List of (x, y) points.
- **Epsilon:** Maximum distance between two samples to be considered in the same cluster.
- **Minimum samples:** Minimum number of points required to form a dense region.

Output:

- **Final clusters:** List of the raw coordinates attached with the cluster's label to each coordinate.
- **Noise points** are discarded and not used in the result.

After running DBSCAN algorithm, based on the defined epsilon and minimum samples (min_samples), there are three types of points:

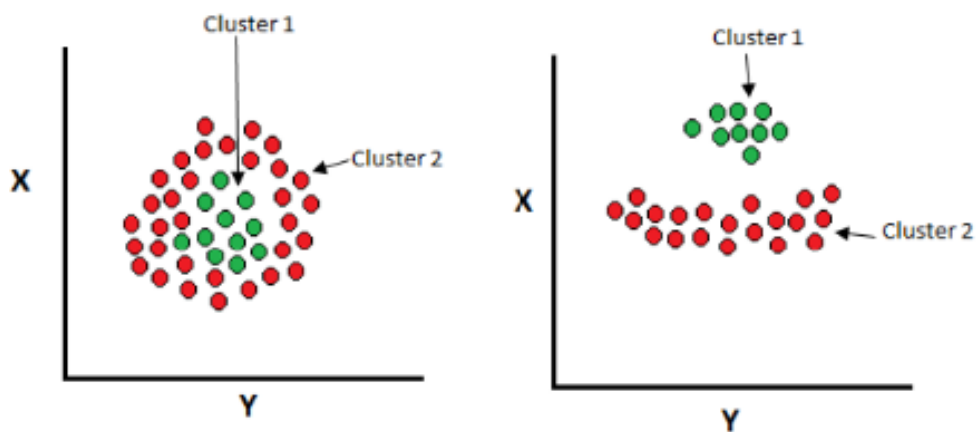
1. **Core Points:** A data point is said to be a core point if there are at least min_samples points within epsilon radius.
2. **Border Points:** A data point is said to be a border point if it has a core point within epsilon radius but has fewer than min_samples points within the epsilon radius.
3. **Noise/Outlier Points:** A data point is said to be a noise/outlier point if it is neither a core point nor has a core point within epsilon radius.



Algorithm:

1. Iterate over all points from the data and classify each one as core/border/outlier.
2. Randomly select a core point that isn't assigned to a cluster and assign it to a new cluster. Iterate through its neighbors (points within epsilon radius) and assign them to the cluster. If the neighbor point is a core point, recursively continue this step to its neighbors.
3. Repeat step 2 until no more core points are left.
4. Traverse through the unvisited points. If they are not part of any cluster, then mark them as outlier/noise.

DBScan Clustering



Optical Flow – Farneback’s Algorithm

In general, Optical Flow is the pattern of apparent motion of objects in a visual scene that is caused by the relative motion between an observer and a scene. The main idea of Farneback’s method is to approximate the intensity of some neighbors of each pixel with a quadratic polynomial:

$$(*) I_1(x) = x^T A_1 x + b_1^T x + c_1$$

as a function of the displacement by following the optical flow constraint equation:

$$I_2(x) = I_1(x - d)$$

and by simply Constructing a modified function of the new intensity we got new coefficients:

$$I_2(x) = (x - d)^T A_1 (x - d) + b_1^T (x - d) + c_1 = x^T A_1 x + (b_1 - 2A_1 d)^T x + d^T A_1 d - b_1^T d + c_1 \stackrel{(*)}{=} x^T A_2 x + b_2^T x + c_2$$

And by equating the coefficients in the quadratic polynomials we could extract the displacement:

$$A_2 = A_1 ; b_2 = b_1 - 2A_1 d ; c_2 = d^T A_1 d - b_1^T d + c_1 ; 2A_1 d = -(b_2 - b_1)$$

$$d = -\frac{1}{2} A_1^{-1} (b_2 - b_1)$$

Our calculations are valid only if A_1 is invertible, but that’s a fair assumption because that happens only in smooth regions and in edges, but for our purpose of detecting the hand which is relatively large to the frame and with texture, we shouldn’t face these problems.

Because we use an approximation of 2nd degree we need to use a gaussian pyramid to comply with the assumption that the movement is between near pixels hence we can perform the approximation.

Input:

- **Previous and next frame** – Used to calculate the difference (motion) between the images.
- **Pyramid scale** = 0.5 – Down-sample rate for the Gaussian pyramid used to achieve the assumption in optical flow that the motion is within neighbor pixels.
- **Levels** = 3 – Number of pyramid layers used to capture coarse motion.
- **Window Size** = 21 – The larger the window size, the more robust the algorithm is to noise, and provide fast motion detection, though gives blurred motion fields. Since we only need an approximate hand position, we rather be robust to noise than have the specific motion field.
- **Iterations** = 3 – Number of iterations to be performed at each pyramid layer.

Output:

- **Flow Image** – Image where each pixel value is the motion vector.

At the last lectures we learnt about optical flow in general and Lucas-Kanade sparse optical flow in particular. Instead of implementing an optical flow algorithm from scratch, we used open-cv's options. Their LK algorithm calculates the optical flow using LK by tracking features. After experimenting we couldn't find robust features of the hands that remain between frames hence unable to track them between frames, and because our main goal is to locate the hand's center of mass, we chose to use Farneback's dense optical flow, which computes motion for all pixels in the image. By leveraging this feature and assuming that while the hand is in motion, most of the board remains stable, we applied a small threshold to eliminate minor noise. This approach resulted in a motion field primarily composed of the hand's movement. Using this data, we were able to create a bounding box to locate the hand's center of mass accurately .

SSIM Score

Structural Similarity Index Measure (SSIM) is a method used to measure the similarity between two images. It is a perception-based method rather than a pixel value difference-based method as MSE. The SSIM between two image patches x, y is defined as follows:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

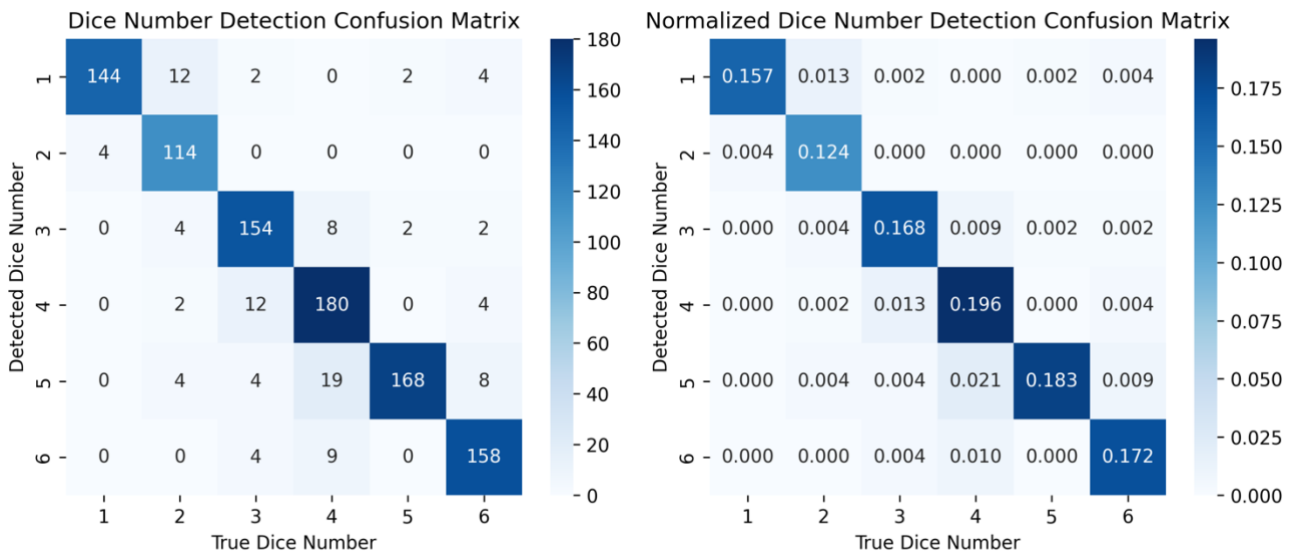
- $\mu_x, \mu_y = \text{mean intensity (luminance)}$
- $\sigma_x^2, \sigma_y^2 = \text{variance (contrast)}$
- $\sigma_{xy} = \text{covariance (structure)}$
- $C_1, C_2 = \text{small constants to stabilize the division}$

It's values mainly range between $[0,1]$ where 0 means no similarity and 1 means perfect similarity. Its advantage over pixel value difference-based methods is that it is invariant to lighting changes to a certain point, where the latter is completely variant making it a more robust measurement between frames as we use it in the corner matching.

Analysis – Dice Number Detection

In the Analysis chapter we decided to analyze our dice number detection algorithm. We've analyzed the die number of 918 different dices we detected across multiple consecutive frames captured during a real gameplay.

Firstly, we'll show the confusion matrix of our detections using our computer vision algorithm as stated in the game flow chapter:

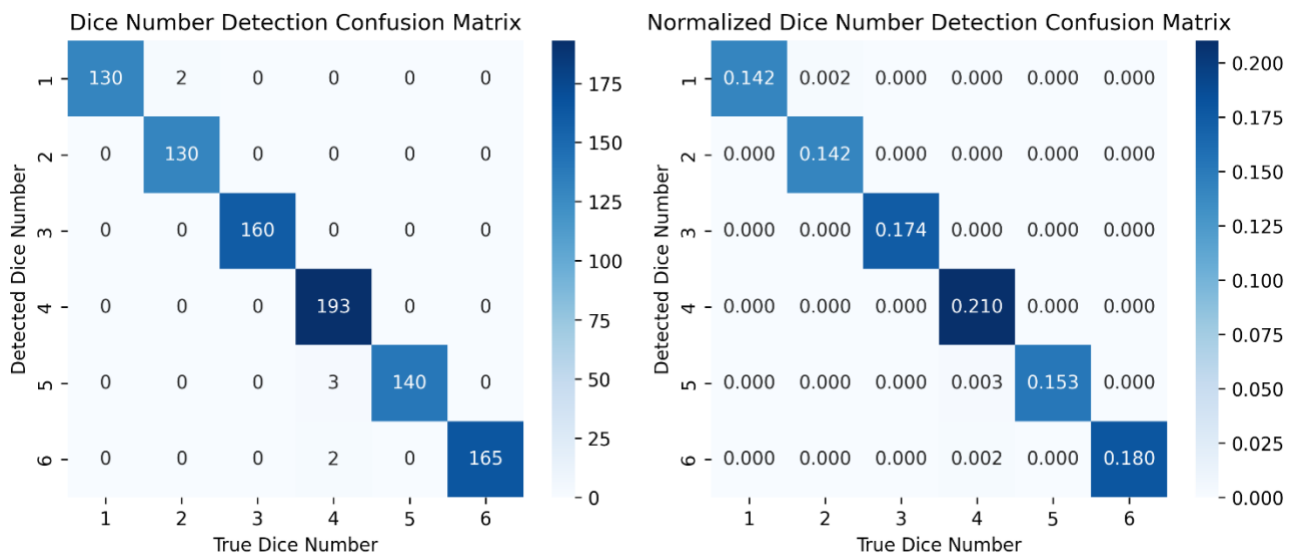


As we can see from the confusion matrix, our computer vision algorithm successfully detected the dice number for an accuracy of **88.5%** of the cases.

We can also see that from the dice we falsely detected, 63% of the false detections were a difference in 1 number (represented as the diagonals adjacent to the main diagonal), Hence our detection algorithm mainly fails by falsely detecting ± 1 dot and less by a noisy image.

When analyzing the dice number detection during gameplay, we saw that most of the false detection occurred due to a noisy dice detection in a single frame ruining the detection of the number of dots. The uniqueness of having most of the false detections in a single frame followed by correct detections in the following frames enabled us to add a logical statement to our algorithm such that we decide the dice numbers were changed only if for at least 2 consecutive frames we detected the same dice numbers, else keep the old numbers we detected. Since our computer vision algorithm runs in real-time speed (7.5 FPS) the logical statement isn't shown to a player's perspective playing the game.

After adding the logical statement we've retested our algorithm and received the following confusion matrix:



As we can see, utilizing the logical sense of the real-world problem, we were able to optimize our algorithm to an accuracy of **99.2%**!

We can see that the remaining false detections occur for a difference of 1 dot and for consecutive frames implying that sometimes there is a noisy light change or the dice and checkers position configuration which our algorithm might have difficulty.

Since the false detections occurred for at most 3 consecutive frames which is less than half a second with our FPS configuration, the algorithm is robust enough for a player's perspective and the false detections do not affect the gameplay.

Conclusions

Throughout the development of the project, we constructed a precise, multi-stage image processing pipeline aimed at real-time analysis of a physical backgammon game. Each stage built upon the success of the previous one, requiring careful design, calibration, and validation. Below are the main insights and conclusions derived from each core component:

Board Detection:

We found that accurate board detection is highly sensitive to edge clarity. To achieve consistent results, we applied a series of preprocessing steps. Despite this, light reflections and interfering hands frequently disrupt detection. To maintain detection of the board, we introduced a mechanism that constructs the board by its corners and a frame-to-frame tracking mechanism, allowing us to reuse previous board detections and ensure temporal consistency.

Grid Creation:

Assuming the board is empty revealed a major limitation – any presence of a checkers blocks triangle detection. Therefore, we computed the grid only once at the beginning of the game. Triangle detection was stabilized through compactness filtering and duplicate removal based on centroid distance, ensuring accurate tile definition.

Checker Detection:

We realized that the unique texture of the checkers is a disadvantage since it varies due to lighting constraints. Hence a threshold solution that unifies the color of the checkers is the solution. Basic color thresholding was insufficient and required an adaptive approach. We analyzed histograms across multiple frames to derive an adaptive threshold technique for each checker color. The next challenge was to split the checkers apart. Since the checkers are adjacent to one another in backgammon, we combined template matching with DBSCAN clustering to reliably extract each checker's center.

Dice Detection:

Due to the visual similarity between white dice and white checkers, we first removed detected checkers from the image. Then, by analyzing contour area, aspect ratio, and minimum bounding box rotation, we isolated the dice. Dot detection was achieved using template matching and DBSCAN clustering. We observed a significant improvement in dice value accuracy only after adding a temporal verification mechanism across multiple frames.

Hand Detection:

Fast hand movement and skin tones like the board ruled out color-based segmentation. We instead used dense optical flow between a clean background frame and the current frame to detect motion. We found that the hand's position relative to the board allowed us to determine the dice thrower's identity, using logic based on the dice's status, coverage, and hand movement direction.

Integration Between Computer Vision and External Game Logic:

The output of our system is fundamentally different from the expected input of existing game management applications. While standard applications are built so that players make moves through a controlled interface, thereby preventing illegal moves altogether, our system observes physical gameplay where players act freely. This means the system must **detect** illegal moves after they occur. As a result, we couldn't rely on any existing game engine and had to implement the entire game logic manually. This included move validation, board state tracking, error detection, and full synchronization between the visual recognition pipeline and the actual game progression.

Full System Integration:

We found that consistency between pipeline stages is critical: an early-stage error propagates downstream. Therefore, each module includes internal validation by various techniques.

In conclusion, the project required us to practically apply theories and tools learned in class, while accounting for the complexities of real-time processing in an uncontrolled physical environment. Dealing with noise, variable lighting, unpredictable motion, and integrating computer vision with external logic made the task particularly challenging. Nonetheless, the experience was enjoyable, enriching, and highly rewarding.



Bibliography

- [1] Introduction to Image Processing – Ben Gurion University.
- [2] https://en.wikipedia.org/wiki/Gaussian_blur.
- [3] https://en.wikipedia.org/wiki/Median_filter.
- [4] Yao, J. (2013). *Median Filtering and Morphological Filtering*. Electrical Engineering, New York University.
- [5] Canny, J. (1986). "A Computational Approach to Edge Detection". IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8(6), 679–698.
- [6] https://en.wikipedia.org/wiki/Canny_edge_detector.
- [7] https://docs.opencv.org/3.4/db/df6/tutorial_erosion_dilatation.html.
- [8] https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html.
- [9] https://docs.opencv.org/3.4/d4/d73/tutorial_py_contours_begin.html.
- [10] https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html .
- [11] https://en.wikipedia.org/wiki/Affine_transformation .
- [12] https://en.wikipedia.org/wiki/Projective_geometry.
- [13] https://docs.opencv.org/4.x/da/d0c/tutorial_bounding_rects_circles.html.
- [14] <https://en.wikipedia.org/wiki/DBSCAN>.
- [15] <https://www.geeksforgeeks.org/opencv-the-gunnar-farneback-optical-flow/>.